

MAVEN Demonstration Plan

MAVEN Demonstration Plan — Detailed Steps & File Locations

1) Show lifecycle sequence

Goal: Demonstrate Maven phases and the order they run.

Steps (live demo)

1. Open terminal / command prompt at project root (folder that contains pom.xml).
2. `cd /path/to/DemoblazeAutomation`
3. Run the lifecycle phases in order (clean → validate → compile → test):
4. `mvn clean validate compile test`

Or run a single combined command:

`mvn clean test`

5. Watch console output. Stop and point out phase headings ([INFO] --- maven-clean-plugin:..., [INFO] --- maven-compiler-plugin:..., [INFO] --- maven-surefire-plugin:...).

Files & locations to show

- pom.xml — (project root) shows plugin configuration and dependencies.
- target/ — created after compile & test phases.
 - Compiled classes: target/classes/ (for main code)
 - Test classes compiled: target/test-classes/
- Surefire results: target/surefire-reports/ (XML/TXT)
- TestNG HTML report: test-output/index.html (if TestNG writes it)

Expected output to point out

- BUILD SUCCESS (if tests pass)
- Console summary: Tests run: X, Failures: Y, Skipped: Z
- The target/ directory created/updated.

Troubleshooting

- If build fails on compile: show compiler error and fix code.
- If test runs no tests: verify testng.xml path or that tests are under src/test/java.

2) Show auto-dependency download (delete .m2 and rebuild)

Goal: Explain Maven Central & local cache behavior; show how Maven downloads dependencies.

Steps (live demo)

1. Close IDEs that might use .m2.
2. **Option A (safe, demo without removing everything):** Move local copy of the project's dependencies folder for demo (recommended in class).
Option B (real clean): Delete artifacts for the demo project or entire .m2 repo (careful on shared machines).
3. # Option B: delete project group folder from .m2

4. `rm -rf ~/.m2/repository/com/yourcompany/yourartifact`
5. # Or to delete complete .m2 (only on disposable environment)
6. `rm -rf ~/.m2/repository`
7. Run:
8. `mvn clean test`

Maven will show download logs: Downloading from central: <https://repo.maven.apache.org/>...

Files & locations to show

- Local Maven repo:
 - Linux / macOS: `~/.m2/repository/`
 - Windows: `%USERPROFILE%\m2\repository\`
- Show downloaded jars after run: e.g.
- `~/.m2/repository/org/seleniumhq/selenium/selenium-java/4.xx/selenium-java-4.xx.jar`

Expected output to point out

- Console lines: Downloading from central ... and Downloaded: ...
- Files appear under `~/.m2/repository/<groupId path>/...`

Troubleshooting

- If downloads fail: network/proxy issues — show Maven settings `~/.m2/settings.xml` for proxy config.
- If permission denied: instruct to run as same user or use local temp .m2.

3) Run mvn package — Generate JAR (artifact creation)

Goal: Show artifact packaging and where the artifact is stored.

Steps (live demo)

1. From project root:
2. `mvn clean package`
3. After success, locate the artifact.

Files & locations to show

- target/ directory:
 - Artifact: `target/<artifactId>-<version>.jar` or `.war`Example:
 - `target/DemoblazeAutomation-1.0.jar`
- `pom.xml` → `artifactId` and `version` fields determine file name.

Expected output to point out

- Console: Building jar: `../target/DemoblazeAutomation-1.0.jar`
- File exists: `ls -l target/`

Troubleshooting

- If packaging fails: check maven-compiler-plugin errors, missing resources, or incorrect packaging type (<packaging>jar</packaging> in pom.xml).

4) Integrate with Jenkins — pipeline executes lifecycle (CI/CD link)

Goal: Show how Jenkins triggers Maven build and displays test results.

Minimal Jenkins declarative pipeline (Jenkinsfile) example

```
pipeline {
  agent any
  stages {
    stage('Checkout') {
      steps {
        checkout scm
      }
    }
    stage('Build & Test') {
      steps {
        sh 'mvn clean test'
      }
    }
    stage('Package') {
      steps {
        sh 'mvn package'
      }
    }
    stage('Publish Reports') {
      steps {
        junit '**/target/surefire-reports/*.xml'
      }
    }
  }
  post {
    always {
      archiveArtifacts artifacts: 'target/*.jar', allowEmptyArchive: true
      publishHTML([allowMissing: true, alwaysLinkToLastBuild: true, keepAll: true, reportDir:
'target/site', reportFiles: 'surefire-report.html', reportName: 'Surefire Report'])
    }
  }
}
```

Steps (live demo)

1. Push project to GitHub (or local Git)
2. Create Jenkins job (pipeline) and point to repo, or create a Multibranch Pipeline.
3. Run job → watch stages run.
4. Open **Blue Ocean** or classic job console to view stage logs.
5. After job success, view **Test Result Trend** and **Archived artifacts**.

Files & locations to show

- Jenkins workspace: <JENKINS_HOME>/workspace/<jobname>/ (on Jenkins server)
- Artifacts archived under job build page (Artifacts link)

- Test results: Jenkins parses `**/target/surefire-reports/*.xml`

Expected output to point out

- Pipeline stages passing
- Jenkins displays number of tests passed/failed
- JAR artifact archived on Jenkins (link to download)

Troubleshooting

- If Jenkins cannot run mvn — ensure Maven installed on Jenkins global tools and PATH.
- If results not shown — ensure junit post step path matches `target/surefire-reports/*.xml`.

5) Explain .m2 caching — show local repo & dependency reuse concept

Goal: Explain benefit of caching and how artifacts are reused.

Steps (live demo)

1. Show `~/.m2/repository/` folder in explorer or terminal.
2. `ls -la ~/.m2/repository/org/seleniumhq/selenium/selenium-java/4.xx/`
3. Run mvn clean test again — note Maven will NOT re-download dependencies already in `.m2`.
4. Show timestamps on files in `.m2` (they don't change) vs `target/` (new each build).

Files & locations to show

- Local repo path: `~/.m2/repository/`
- Example downloaded dependency:
`~/.m2/repository/org/testng/testng/7.11.0/testng-7.11.0.jar`

Expected output to point out

- Faster builds once dependencies are cached.
- Space usage trade-off: `.m2` can grow — explain housekeeping policies.

Troubleshooting

- If dependency versions conflict: show mvn `dependency:tree` to visualize dependency graph and conflicts.

Additional Useful Commands (copy-paste for demo)

- Run only tests specified in `testng.xml` via Surefire:
- `mvn -Dsfire.suiteXmlFiles=src/test/resources/testng.xml test`
- Generate surefire HTML report:
- `mvn surefire-report:report-only`
- # result at `target/site/surefire-report.html`
- See dependency tree (useful when teaching `.m2` caching and conflicts):
- `mvn dependency:tree`

- Clean local cached artifacts for a specific group:
- `rm -rf ~/.m2/repository/com/yourcompany`
- Deploy artifact to remote (requires distributionManagement and server credentials in `~/.m2/settings.xml`):
- `mvn clean deploy`

Hands-on Classroom Notes & Tips

- **Prepare environment beforehand:** install Maven, Java, Chrome/ChromeDriver or WebDriverManager, and Jenkins (or use a shared Jenkins server).
- **Use small example tests** (3–5 tests) for demo speed.
- **Show both CLI and IDE runs:** CLI demonstrates Maven lifecycle; IDE demo (Run as TestNG) is quick check but emphasize CI uses CLI/Maven.
- **Safety with .m2:** avoid deleting full .m2 on shared machines; instead delete specific group/artifact for demo clarity.
- **Network/proxy handling:** show `~/.m2/settings.xml` if behind corporate proxy.
- **Artifacts & security:** explain settings.xml server credentials (do not show real credentials).

Quick FDP Script (what you say during demo)

1. “We’ll run the Maven lifecycle: validate → compile → test. I’ll run `mvn clean test` now.” (Run it.)
2. “See `target/` created — compiled classes are there, and Surefire created reports under `target/surefire-reports/`.” (Open folder.)
3. “Now I’ll remove the local cache for our project and rebuild to show Maven downloading dependencies.” (Delete and run `mvn clean test`.)
4. “Package the artifact so we can share it.” (Run `mvn package`, show `target/*.jar`.)
5. “We’ll push to Git and use Jenkins to execute the same lifecycle automatically.” (Show Jenkinsfile and run job.)
6. “Finally, .m2 caching means subsequent builds are faster because dependencies are reused.” (Run `mvn test` again and show no downloads.)